Target Window ID / Devices
Updates layout image_path, w, h, dpi
Direct CRUD on layouts, zones, settings
Import / Export JSON, TOML, Bundles
Persists sprint_key, rate_cap, pps
250ms debounced & immediate updates
1000ms telemetry & psutil tree poll
GUI Import / Export
Hosts canvas & syncs Inspector
Spawns Engine & dispatches IPC (0x03)
_notify_reload() -> ON_RELOAD
layout_saved -> ON_RELOAD
Transactional migration / read
save_with_zones() + apply_image_params()
2.0s live window_info & screen bounds sync
Publishes .touch2key_runtime (1Hz)
Reads active Layout, Zones, Bezels, Settings
VKB_STRUCT (<BH) pipe
Spawns VKB (0x9906)
Feeds sorted self._tiers
True (Menu Mode)
False (In-Game: 2-finger tap ignored)
IDLE / REJECTED (Replay)
Yes (Active Owner)
No (Unowned Contact)
pipeline.process()
Claimed on DOWN
Yes (DOWN)
No (Unowned MOVE/UP)
Unclaimed on DOWN
Yes (Menu Mode: self.is_visible)
mouse_move_abs & left_click
_acc_x, _acc_y
Keys / Clicks / Hotkeys
mouse_move_rel(idx, idy)

# TOUCH2KEY FULL-STACK ARCHITECTURE: GUI, CLI SCRIPTS, SQLITE STORE, TELEMETRY & 3-PHASE ENGINE PIPELINE (ADB -> OS DRIVERS)

## LAYER 1: BOOTSTRAP, CLI SCRIPTS, ADB UTILITIES & DUAL-MODE DIALOGS (modules.scripts.* & modules.cli.*)

### 1A. Environment Preflight & ADB Discovery
touch2key-preflight (preflight.py) | touch2key-adb (check_adb.py)

| | |
|---|---|
| **check_adb()** | Checks BIN_FOLDER/adb (ADB.exists()) -> fallback shutil.which('adb') -> resolves absolute binary path |
| **check_driver()** | Win32: Interception().valid check \| Linux: Path('/dev/uinput').exists() verification |
| **Failure Reporting** | Returns bool; logs actionable remediation (touch2key-setup / sudo touch2key-setup / PC reboot) |

### 1B. Platform Bootstrap & Uninstaller
touch2key-setup (setup.py) | touch2key-uninstall (uninstall.py) [--gui, -y, --no-restart, --purge, --purge-all]

| | |
|---|---|
| **Privilege Elevation** | Win32: IsUserAnAdmin() -> ShellExecuteW('runas', sys.executable, '-m ...') \| Linux: pkexec (GUI) or sudo (CLI) |
| **OS Driver Setup** | setup_windows() installs Interception & platform-tools \| setup_linux() writes UDEV_RULE_PATH & reloads udevadm |
| **Desktop Shortcuts** | Win32: PowerShell WScript.Shell -> Touch2Key.lnk (pythonw.exe -m modules.gui.app) \| Linux: touch2key.desktop (0o755) |
| **Uninstall & Purge** | store.close() -> _kill_adb() (taskkill/pkill) -> install-interception.exe /uninstall or udev rm -> rmtree(BIN/DATA/LOGS/DIAGNOSTICS) |

### 1C. ADB Screen Capture & Wireless Handshake
touch2key-capture (adb_screen_capture.py) | touch2key-wireless (wireless_connect.py & wireless_connect_dialog.py)

| | |
|---|---|
| **Device Metrics** | get_adb_device() -> _get_device_dpi() (get_dpi() or 'adb shell wm density', fallback BASELINE_DPI) + get_rotation() + rotate_resolution() |
| **ADB Screencap** | adb shell screencap -p /data/local/tmp/temp_cap.png (30s timeout) -> adb pull (20s) -> finally: adb shell rm |
| **PIL & Layout Sync** | Image.open() -> img.load() (releases Win32 file lock) -> img.save(dpi=(dpi,dpi)) -> store.layouts.update(id, image_path, w, h, dpi) |
| **Wireless ADB** | CLI: wireless_connect(continuous=True) \| GUI: WirelessConnectDialog + WirelessConnectWorker(QThread) polling every 1.0s (10x0.1s) |

### 1E. CLI Layout, Profile & Settings Manager
touch2key-manage (modules/cli/manage.py) — Argparse Flags & Interactive Loop

| | |
|---|---|
| **Profile & Zone CRUD** | --list, --filter, --sort, --new [--dims W H DPI], --set-active, --duplicate, --rename, --delete, --clear-zones, --reset-zones |
| **Zone Inspector CLI** | --list-zones, --set-bezels ID TOP BOT, --edit-zone ZID [--zone-priority/pointer/sens-x/sens-y/deadzone/hysteresis], --reset-zone, --delete-zone |
| **Directionals & Image** | --show-directional-keys, --edit-layout-directionals ID --set-key {up,left,down,right}, --set-image ID PATH (calls apply_image_params) |
| **Global Tuning & I/O** | _normalize_key_seq() ('k1,k2' replace, '+k1' append, '+wasd'/'+arrows' presets, 'clear' wipe), --import/export-json/toml/bundle, --wipe-database |

### 1D. Dual-Mode Window Selector & Key Capture
select_window() | capture_sprint_key() & capture_toggle_hotkeys()

| | |
|---|---|
| **Window Enumeration** | _gather_window_rows(WindowManager): find_visible_windows() -> filters width > 0 & height > 0 -> [wid, title, class, l, t, w, h] |
| **CLI Selector** | 1.0s auto-refresh table via AppLogManager.console_write; 50ms non-blocking poll (Win32 msvcrt.getch \| Unix tty.setcbreak + select) |
| **GUI Selector** | ListApp(QDialog): QTableWidget with 1000ms QTimer diff-sync (added_ids, removed_ids, current_ids) & UserRole wid storage |
| **Capture Key/Toggles** | GUI: KeyCaptureDialog & KeySequenceRow vs. CLI: capture_toggle_hotkeys() using blocking msvcrt/tty prompt inputs |

## LAYER 2: PYSIDE6 GRAPHICAL USER INTERFACE, DEBOUNCED SETTINGS & MATPLOTLIB PLOTTER (modules.gui.*)

### 2A. MainWindow & EngineProcessController
touch2key-gui (modules/gui/main_window.py) — Process Lifecycle & Thread-Safe Event Bridge

| | |
|---|---|
| **EngineProcessController** | Manages engine multiprocessing.Process, holds lock for (window_id, window_title, k_id, m_id), pipes runtime IPC commands |
| **Page Navigation & Dock** | QStackedWidget hosting DashboardPage, LayoutStudioPage, SettingsPage + collapsible Log Dock fed by QtLogHandler |
| **Close & Action Guards** | save_all_pages(action) invokes LayoutStudioPage.save_state() (Save/Discard/Cancel prompt) & SettingsPage._flush_pending() |

### 2B. DashboardPage (Live Telemetry & Session Summary)
modules/gui/pages/dashboard_page.py — 1000ms QTimer (Active Only While Visible)

| | |
|---|---|
| **Dispatcher Hooks** | Registers ON_RELOAD -> _reload_requested Signal & ON_TARGET_WINDOW_CHANGE -> _target_window_changed Signal (queued to GUI thread) |
| **Session Summary** | Displays active Layout name/ID, non-bezel zone count (count_by_layout()), Target Window title/HWND, Toggle Hotkeys, Sprint, k_id/m_id |
| **Snapshot Telemetry** | read_snapshot() -> renders rate_hz, cap drift, event counts \| labeled_processes() links PIDs |
| **Accurate Memory** | psutil.cpu_percent() normalized by cpu_count(); psutil.memory_full_info().uss extracted to perfectly match Task Manager Private Working Set |

### 2E. SettingsPage (Hardware Rebind, Key Filters & Debounced Writes)
modules/gui/pages/settings_page.py — 250ms Coalesced SpinBox Timer with Exponential Backoff

| | |
|---|---|
| **Hardware & Window** | Select Target Window -> ON_TARGET_WINDOW_CHANGE \| Win32 Detect & Rebind Hardware (select_keyboard_then_mouse) -> ON_DEVICES_CHANGE |
| **Event Filter Captures** | KeyCaptureFilter (single stroke) & ContinuousSequenceFilter (KeySequenceRow for toggle_hotkeys & typematic exclusions; intercepts OS events & blocks Tab focus) |
| **Debounced SpinBoxes** | _queue_save() batches sensitivity_x/y, deadzone, hysteresis, adb_rate_cap, pps, typematic_delay_ms/rate_hz -> _flush_pending() every 250ms |
| **Immediate Toggles** | left_handed, floating_joystick, anchored_joystick, typematic_enabled, two_finger_tap_enabled, bezel_toggle_enabled -> immediate store.settings.update() + ON_RELOAD |

### 2C. LayoutStudioPage (3-Pane Editor & Pipeline Inspector)
modules/gui/pages/layout_studio_page.py & touch2key-plot (studio.py)

| | |
|---|---|
| **Left Sidebar (Profiles)** | Sort QComboBox (synced to profile_sort_order), Ctrl+F filter, QListWidget with ★ Active, ● <24h recency, * dirty badge; Set Active/New/Rename/Duplicate |
| **Center Toolbar** | Select Image / Capture Screenshot (_apply_image -> get_image_params), Toggle Overlays (F4), Add Circle (F6), Add Rect (F7), Cancel (Esc), Save (Ctrl+S / Ctrl+Shift+S) |
| **Right Inspector (Stack)** | Page 0: Top/Bottom Bezel SpinBoxes (MIN..MAX_BEZEL_DP) + Reset All Zones \| Page 1: Zone Priority, Ignore AppSettings, Sens X/Y, Deadzone, Hysteresis, Pointer |
| **Unsaved Guards** | _confirm_unsaved() prompts before profile switch, duplicate, import, export, or image change; _on_inspector_value_changed updates PipelineConfig JSON |

### 2D. LayoutsPlotterWidget (Matplotlib QTAgg Blit Canvas)
modules/gui/widgets/layouts_plotter_widget.py — Interactive Geometry & Artist Manager

| | |
|---|---|
| **PlotterState FSM** | IDLE \| COLLECTING (3-pt Circle circumcenter / 4-pt Rect bbox) \| WAITING_FOR_KEY \| WAITING_FOR_WHEEL_INDICATOR \| DELETING \| CONFIRM_DELETE_ALL |
| **Blit Drag Architecture** | Single mpl_connect per event dispatches to _DraggableLabel, _DraggableCircle (rim resize vs drag), _DraggableRectangle (4 corners + 4 edges + drag) |
| **Wheel & Sprint Coupling** | MouseWheelIndicators (4 directional overlays 0:Up, 1:Left, 2:Down, 3:Right; _MouseWheelDragManager vetoes wheel drag); radially constrains Sprint outside wheel |
| **Atomic DB Save** | save_to_database() -> _build_zone_rows() -> store.layouts.save_with_zones() -> store.settings.update(active_layout_id) -> layout_saved Signal -> ON_RELOAD |

## LAYER 4: SQLITE SINGLE SOURCE OF TRUTH, CONFIG I/O & TELEMETRY SNAPSHOT (modules.database.* & modules.runtime_info)

### 4C. Runtime Telemetry Snapshot & CLI Stats
modules/runtime_info.py | touch2key-stats (stats.py)

| | |
|---|---|
| **Snapshot Payload** | JSON file (.touch2key_runtime): pid, updated_at, rate_hz, rate_status, rate_cap_hz, pps_threshold, menu_mode, downs/moves/ups/all, respawns, workers, window_info, screen_width/height, window_lost |
| **Consumers** | Read every 1.0s by DashboardPage (_update_telemetry), every 2.0s by Visualizer (attach), and by touch2key-stats (-i, --once, --json, --clear) |

### 4B. Config I/O & Legacy Migration Engine
modules/database/config_io.py & legacy_migration.py

| | |
|---|---|
| **Import Pipeline** | import_any() auto-detects .json layouts (migrate_json_layout), .toml settings (migrate_toml_config), or bundle directories |
| **Export Pipeline** | export_layout_json(), export_settings_toml(linked_json_path), export_bundle() writing portable TOML + JSON pairs to data/profiles/ |

### 4A. SQLite Relational Store (touch2key.db)
modules/database/ — Foreign-Key Cascaded Repositories (store.settings, store.layouts, store.zones)

| | |
|---|---|
| **SettingsRepository** | Singleton AppSettings row: handedness, joystick modes, sens_x/y, deadzone, hysteresis, rate_cap, pps, typematic_*, toggle_hotkeys, active_layout_id |
| **LayoutsRepository** | Layout rows (width, height, dpi, mouse_wheel_radius, sprint_distance, image_path); apply_image_params() atomically rescales all non-bezel zones on resolution change |
| **ZonesRepository** | Zone rows (scancode, zone_type, cx, cy, r, x1..y2, priority, pointer, ignore_app_settings, pipeline_json); auto-heals Top (0x9905) & Bottom (0x9906) Bezels |

## LAYER 3: DIAGNOSTIC & RUNTIME IPC OVERLAYS (modules.gui.overlays.*)

### 3A. Live Touch & Active Layout Visualizer
touch2key-visualizer (visualizer.py) — Single-Instance Fullscreen Translucent QGraphicsView

| | |
|---|---|
| **Process Registration** | check_single_instance('Touch2Key_Visualizer') -> register_process(VISUALIZER_RUNTIME_FILE) -> atexit unregister_process |
| **Visual Feedback** | mouseMoveEvent draws interpolated bresenham() trail rects; key/mouse press triggers active hitbox highlights and expiring touch ripples |
| **Live Coordinate Sync** | Monitors toggle_hotkeys to toggle cursor visibility; every 2.0s reads telemetry window_info & screen_width/height to scale and offset layout zones to the active target window (or fullscreen viewport fallback) |

### 3B. Low-Latency Virtual Keyboard (VKB)
touch2key-keyboard — Spawned via Bottom Bezel (0x9906) or CLI

| | |
|---|---|
| **Lifecycle & Tracking** | Managed by Engine.toggle_virtual_keyboard(); PID published in runtime snapshot under workers['vkb'] |
| **Direct IPC Stream** | Serializes virtual keystrokes into VKB_STRUCT (<BH: action, scancode) and pipes directly to Engine Mapper for zero-focus-loss injection |

## ENGINE PHASE 1: PIPELINE CONSTRUCTION, RELOAD TRANSACTION & DETERMINISTIC 5-KEY TIER SORTING

### 5.1 Initialization & Reload Triggers
Engine.start_headless() | MapperEvent(action='ON_RELOAD')

| | |
|---|---|
| **Initial Boot** | store.settings.get() -> clamps rate_cap [30..1000Hz], pps [10..500], binds callbacks, starts WindowManager self-healing tracker |
| **ON_RELOAD Sources** | CLI [F5] Handedness \| CLI [F6] Reload \| CLI Arrow Key 250ms Commit \| GUI Settings/Studio MapperEvent \| IPC (0x03) |
| **Debouncer** | MapperEventDispatcher._debounce_loop (10ms tick); immediate synchronous delivery if cooldown == 0 |

### 5.2 Atomic Reload Transaction (Engine._on_reload under self.lock)
Layout Snapshot Sync & State Boundary Sanitation

| | |
|---|---|
| **LayoutLoader** | load_layout() extracts SQLite snapshot and normalizes zones into oriented device pixels (dev_w, dev_h, dev_dpi) |
| **Hotkeys Guard** | Under _adjust_lock: syncs rate_cap & pps from DB only if not _settings_dirty |
| **TouchReader** | touch_reader.on_reload() & touch_reader.set_rate_cap(self.rate_cap) |
| **_release_active_inputs()** | _dispatch_generation += 1 \| clears _contact_owners \| resets all active pipelines & releases held keys/buttons |

### 5.3 Submapper Pipeline Reconstruction
Deterministic Rebuild Sequence & Pointer Generation

| | |
|---|---|
| **Mapper.reload()** | Updates device_width, device_height, dpi, toggle_hotkeys_str, pps |
| **BezelMapper** | TOP_BEZEL (0x9905) -> execute_toggle_hotkeys \| BOTTOM_BEZEL (0x9906) -> toggle_virtual_keyboard (Prec: 4) |
| **MouseMapper (Game)** | Game Mode Camera Pipeline: Custom Look Zone or Handedness Half-Screen / AlwaysRegion -> DeltaTransform -> PointerSemantic (Prec: 0) |
| **MouseMapper (Fallback)** | get_fallback_mouse_pipeline(): Universal AlwaysRegion + MousePointer fallback created for Menu Mode cursor dispatch |
| **KeyMapper** | update_bridge_typematics() (<B?IfH) + Button Pipelines (Button + optional PointerSemantic, Prec: 2) |
| **WheelMapper** | Joystick Pipeline: Fixed/Anchored/Dynamic Origin -> JoystickTransform -> DirectionalSemantic (Prec: 1) |

### 5.4 Isolated 5-Stage Pipeline Contract (Pipeline[T])
Region -> Origin -> Constraint -> Transformation[T] -> list[Semantic[T]]

| | |
|---|---|
| **Stage 1: Region** | AlwaysRegion (area=inf) \| CircularRegion (pi*r^2) \| RectangularRegion (w*h) |
| **Stage 2: Origin** | FixedOrigin (static) \| DynamicOrigin (touch DOWN) \| AnchoredOrigin (snap_radius=80.0 default) |
| **Stage 3: Constraint** | NoConstraint \| RadialConstraint (clamp r) \| LeashConstraint (pulls origin if dist > leash_r) |
| **Stage 4: Transform** | IdentityTransform -> UNIT \| DeltaTransform -> Vector \| JoystickTransform -> frozenset[int] |
| **Stage 5: Semantics** | ButtonSemantic (key/mouse) \| DirectionalSemantic (WASD diff) \| PointerSemantic (dx,dy) \| ToggleSemantic (tap) |

### 5.5 Engine._build_pipeline_tiers()
Strict 5-Key Sort & Equivalence Grouping

| | |
|---|---|
| **Key 1: -precedence** | Button (3) > Bezel (2) > Joystick (1) > Mouse/Pointer (0) [allows overlapping buttons to claim touches over bezel zones] |
| **Key 2: -priority** | Custom user priority tier (+200 down to -100) |
| **Key 3: -multi_claim** | allow_multi_claim: True (sorted first) before False (sorted last) |
| **Key 4 & 5** | +region.area (smaller hitboxes first) -> +creation_id (deterministic tie-breaker) |
| **Output: self._tiers** | list[list[Pipeline]] partitioned by identical (precedence, priority, allow_multi_claim) |

## ENGINE PHASE 2: RUNTIME TOUCH INGESTION, TWO-FINGER TAP FSM & CONTACT OWNERSHIP DISPATCH

### 6.1 TouchReader -> Engine._process_touch_event(touch_event)
ADB ABS_MT_* Stream (Synchronized under Engine.lock)

| | |
|---|---|
| **ADB Auto-Adapt** | Queries ABS_MT_POSITION_X/Y & slot capabilities on connect; auto-pauses & resumes on USB/Wi-Fi drop |
| **Event Payload** | TouchEvent(contact_id: int, phase: DOWN \| MOVE \| UP, position: Point(x,y), timestamp: float) |
| **Guard & Telemetry** | Aborts if shutting down \| Records TouchEventCount & increments mapper.event_count |

**Menu Mode Gesture Gate?**

self.is_visible (cursor visible = Menu Mode only) and self.two_finger_tap_enabled

### 6.2 TwoFingerTapTracker.observe(touch_event) - uses time-driven polling to avoid stationary holds
Menu Mode only (cursor visible) | Ambiguity Buffer FSM (duration <= 0.25s, drift <= 25px, sync <= 0.12s)

| | |
|---|---|
| **PENDING** | Buffers candidate event in _buffer and returns early (prevents single-tap leak) |
| **REJECTED** | Replays all buffered events in decision.replay through _dispatch_normal() in order |
| **RECOGNIZED** | Valid synchronized 2-finger tap -> gracefully shuts down the engine process |
| **IDLE** | Unbuffered non-DOWN event -> proceeds to _dispatch_normal(touch_event) |

**1. Is cid in self._contact_owners?**

Ownership is locked at DOWN; MOVE/UP route exclusively to owning pipelines

### 6.3 Route to Owning Pipeline(s)
owners = self._contact_owners[cid]

| | |
|---|---|
| **Process** | for pipeline in owners: pipeline.process(event, output_sink) |
| **Generation Fence** | Aborts immediately if generation != self._dispatch_generation |
| **UP Cleanup** | If event.phase is TouchPhase.UP: self._contact_owners.pop(cid, None) |

**2. Is event.phase == TouchPhase.DOWN?**

New contact arrivals evaluate sorted tiers; unowned MOVE/UP fall through to cursor fallback

### 6.4 Evaluate Sorted self._tiers (DOWN Phase Only)
Strict Deterministic Precedence Claims

| | |
|---|---|
| **Mode Filters** | Skip bezel if not bezel_toggle_enabled \| Skip non-bezel HUD if self.is_visible (Menu Mode) |
| **Claim Test** | pipeline.claims(event) -> _owned_contact is None and region.activates(event) |
| **Game Look Claim** | In Game Mode, MouseMapper's Look Pipeline (Prec: 0) claims camera look contacts |
| **Multi-Claim Rule** | If allow_multi_claim: claim all matching in tier \| Else: break on first claim in tier |
| **Lock Ownership** | If claimed: self._contact_owners[cid] = tuple(claimed) & returns immediately |

**3. Universal Fallback (self.is_visible)?**

Unclaimed contact or unowned MOVE/UP: evaluate desktop cursor fallback

### 6.5 Engine._dispatch_cursor(event)
Menu Mode Absolute Desktop Cursor Fallback

| | |
|---|---|
| **Absolute Coordinates** | output_sink.mouse_move_abs(x, y) + DOWN: mouse_down(M_LEFT) \| UP: mouse_up(M_LEFT) |
| **Fallback Pipeline** | fallback_mouse_pipeline.process(event, output_sink): AlwaysRegion + MousePointer fallback |

## ENGINE PHASE 3: 5-STAGE PIPELINE EXECUTION, FRACTIONAL MOUSE FLUSH & HARDWARE BRIDGE WORKERS

### 7.1 Pipeline.process(event, output_sink)
Isolated 5-Stage Per-Contact Execution

| | |
|---|---|
| **Stages 1 & 2** | DOWN: _owned_contact = cid; origin.begin(pos) \| All: origin.update(pos, constraint); origin_pt = origin.get() |
| **Stage 3: Constraint** | constrained = constraint.apply(origin_pt, pos) \| Computes delta & frame_delta in PipelineContext |
| **Stage 4: Transform** | value = transformation.apply(context) -> UNIT \| Vector(dx,dy) \| frozenset[scancodes] |
| **Stage 5: Semantics** | For semantic in semantics: semantic.process(context, value, output_sink) |
| **UP Teardown** | If phase is UP: origin.end(), transformation.reset(), clears _owned_contact & _prev_position |

### 7.2 BridgeOutputSink & Bezel Callbacks
Sub-Pixel Mouse Accumulator & Action Router

| | |
|---|---|
| **Keys & Clicks** | key_down/up(scancode) \| mouse_down/up(0x9900..0x9904) -> immediate Bridge call |
| **Fractional Move** | mouse_move(dx, dy) -> under _lock: accumulates _acc_x += dx, _acc_y += dy |
| **Top Bezel Callback** | Mapper.execute_toggle_hotkeys(): parse_seq -> key_down stack -> 15-35ms sleep -> LIFO key_up |
| **Bottom Bezel Callback** | Engine.toggle_virtual_keyboard(): spawns/stops VKB process -&gt (suspends typematic while virtual keyboard is active); VKB_STRUCT (<BH) pipe to Mapper |

### 7.3 MouseFlushThread (Engine._mouse_flush_loop)
Rate-Capped Whole-Pixel Dispatcher (1.0 / max(rate_cap, 30.0) s)

| | |
|---|---|
| **flush_mouse_move()** | idx, idy = clamp(round(_acc_x), -32767, 32767) -> bridge.mouse_move_rel(idx, idy) |
| **Remainder & Reset** | Retains fractional _acc_x -= idx, _acc_y -= idy \| Resets to 0.0 when active_touches == 0 |

### 7.4 AbstractBridge -> OS Driver Worker Processes (k_proc & m_proc)
3 Dedicated IPC Pipes + 2 Threads Per Worker Process + Watchdog Respawner

| | |
|---|---|
| **Platform Signatures** | Win32: keyboard_worker(k_pipe, log_q, k_handle), mouse_worker(m_pipe, mb_pipe, log_q, m_handle), NtSetTimerResolution \| Linux: UInput(KEYBOARD_CAP / MOUSE_CAP) |
| **k_proc Main Thread** | Polls k_pipe (15s timeout -> _release_all_keys): unpacks PACK_KEY (<HB, maps E0/LINUX_KEY_MAP) -> key_queue \| unpacks KEY_CONFIG (<B?IfH) |
| **k_proc Injector Thread** | Drains key_queue: DOWN sleeps 40-70ms dwell, UP sleeps 1ms (stretches fast taps) \| Outer loop fires typematic repeat_key after delay_ns at repeat_rate |
| **m_proc Main Thread** | Polls m_pipe: coalesces up to MAX_COALESCE=20 TASK_REL (<Bhh, 0.8-1.2ms dwell) or TASK_WHEEL (<Bhh, 1ms dwell) + TASK_ABS (<Bii, MOUSE_VIRTUAL_DESKTOP) via send_lock |
| **m_proc Button Thread** | Polls mb_pipe (15s timeout -> _release_all_buttons): injects PACK_BUTTON (<Bi) under send_lock, then sleeps 25-40ms DOWN / 1ms UP outside lock (never blocks moves) |